

מבחן תרגול ב-SQL

נושאים: SELECT, WHERE, JOIN, נורמליזציה ואופטימיזציה

תאריך: _____ שם הסטודנט/ית: _____
סה"כ ניקוד: 100 נקודות משך הבחינה: 90 דקות

הוראות: יש לענות בצורה מסודרת וברורה. בשאלות קוד ניתן להשתמש בתחביר SQL סטנדרטי. אם קיימת יותר מתשובה נכונה אחת לשאלת קוד, יש לבחור פתרון יעיל וברור. אין צורך לזכור שמות פונקציות ייחודיות למסד נתונים מסוים, אלא אם צוין אחרת.

סכמת הנתונים לבחינה

בכל השאלות, אלא אם נאמר אחרת, השתמשו במסד נתונים של חנות אונליין:

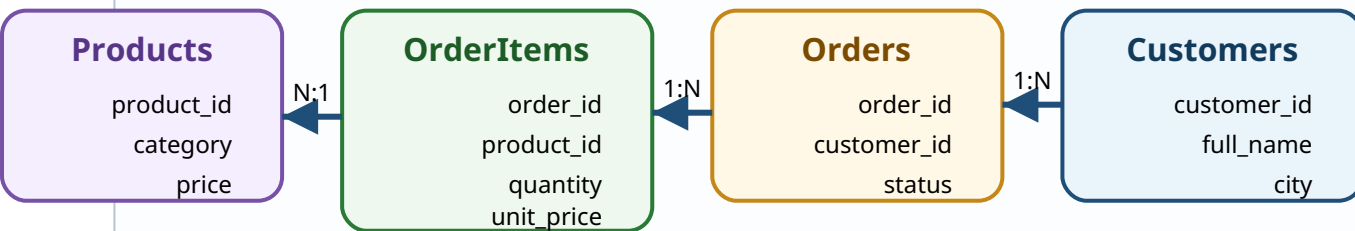
Customers
customer_id PK
full_name
city
join_date

Orders
order_id PK
customer_id FK
order_date
status

OrderItems
order_id FK
product_id FK
quantity
unit_price

Products
product_id PK
product_name
category
price

איור 1 - קשרים בין הטבלאות:



חלק אי - שאלות אמריקאיות (30 נקודות, 3 נקודות לכל שאלה)

1. איזו פקודה מחזירה רק את העמודות full_name ו-city מתוך Customers?

1. `SELECT * FROM Customers`;
2. `SELECT full_name, city FROM Customers`;
3. `WHERE full_name, city FROM Customers`;
4. `GET full_name AND city FROM Customers`;

2. מה תהיה המשמעות של התנאי הבא?

```
WHERE city = 'Tel Aviv' AND join_date >= '2024-01-01'
```

1. לקוחות מתל אביב או לקוחות שהצטרפו מ-2024
2. לקוחות מתל אביב בלבד, ללא קשר לתאריך
3. לקוחות שגם גרים בתל אביב וגם הצטרפו החל מ-2024
4. כל הלקוחות חוץ מתל אביב

חלק אי - שאלות אמריקאיות - המשך

3. איזו שאילתה תחזיר את כל ההזמנות יחד עם שם הלקוח שביצע אותן?

1. `SELECT * FROM Orders JOIN Customers ON Orders.customer_id = Customers.customer_id`;
2. `SELECT * FROM Orders JOIN Products ON Orders.order_id = Products.product_id`;
3. `SELECT * FROM Customers WHERE Orders.customer_id = Customers.customer_id`;
4. `SELECT Orders FROM Customers`;

4. מה ההבדל המרכזי בין INNER JOIN לבין LEFT JOIN?

1. INNER JOIN מחזיר גם שורות שלא התאימו, LEFT JOIN לא
2. LEFT JOIN מחזיר את כל השורות מהטבלה השמאלית, גם אם אין התאמה בטבלה הימנית
3. אין הבדל, אלו מילים נרדפות
4. LEFT JOIN משמש רק לטבלאות עם מפתח ראשי

5. בשאילתה עם WHERE ו-ORDER BY, מה הסדר הלוגי הנכון?

1. קודם מיון ואז סינון
2. קודם סינון ואז מיון
3. שניהם מתבצעים תמיד במקביל
4. ORDER BY חייב להופיע לפני WHERE

6. איזו שאילתה תחשב את סכום המכירות לכל הזמנה לפי OrderItems?

1. `SELECT order_id, SUM(quantity * unit_price) FROM OrderItems GROUP BY order_id`;
2. `SELECT order_id, quantity * unit_price FROM OrderItems GROUP BY unit_price`;
3. `SELECT SUM(order_id) FROM OrderItems`;
4. `SELECT order_id FROM OrderItems WHERE SUM(quantity)`;

7. איזו בעיה נורמליזציה מנסה לצמצם?

1. רק איטיות של שאילתות
2. כפילויות נתונים ותלויות לא תקינות
3. שימוש ב-JOIN

4. כתיבת שמות עמודות באנגלית

8. טבלה שבה לכל לקוח נשמרים השדות product1, product2, product3 מפרה בעיקר את הרעיון של:

1. עמודות חוזרות במקום רשומות נפרדות

2. מפתח זר

3. ORDER BY

4. אינדקס ייחודי בלבד

9. איזה שינוי עשוי לשפר ביצועים של שאילתה שמסננת הרבה לפי customer_id?

1. מחיקת המפתח הראשי

2. הוספת אינדקס מתאים על customer_id

3. החלפת SELECT ב-DELETE

4. שינוי שם הטבלה

10. למה בדרך כלל לא מומלץ להשתמש ב-SELECT * במערכת אמיתית גדולה?

1. כי SQL לא תומך בזה

2. כי זה תמיד מחזיר שגיאה

3. כי זה עלול להחזיר עמודות מיותרות ולהעמיס על הביצועים והתחזוקה

4. כי זה עובד רק עם JOIN

חלק ב' - נכון / לא נכון (16 נקודות, 2 נקודות לכל סעיף)

סמנו נכון או לא נכון. אין צורך לנמק, אלא אם תרצו להוסיף הערה קצרה.

#	טענה	נכון	לא נכון
1	פקודת WHERE משמשת לסינון שורות לפי תנאי.		
2	INNER JOIN מחזיר גם רשומות שאין להן התאמה בין הטבלאות.		
3	אפשר לבצע JOIN בין יותר משתי טבלאות באותה שאילתה.		
4	נורמליזציה יכולה להפחית כפילויות, אך לפעמים מוסיפה צורך ב-JOIN.		
5	מפתח ראשי יכול להכיל ערך NULL.		
6	אינדקס תמיד משפר כל שאילתה, ללא חסרונות.		
7	HAVING משמש לסינון קבוצות לאחר GROUP BY.		
8	LEFT JOIN יכול להחזיר ערכי NULL בעמודות של הטבלה הימנית.		

חלק ג' - שאלות קוד קצרות (24 נקודות)

1. (5 נק') כתבו שאילתה שמחזירה את כל הלקוחות מהעיר Haifa, ממוינים לפי תאריך הצטרפות מהחדש לישן.

2. (5 נק') כתבו שאילתה שמחזירה order_id, order_date ו-full_name עבור כל הזמנה. יש להשתמש ב-JOIN בין Customers ל-Orders.

3. (5 נק') השלימו את השאילתה כך שתחזיר את שמות המוצרים בקטגוריה 'Books' שמחירים גדול מ-50.

```
SELECT product_name, price
FROM Products
WHERE _____;
```

4. (4 נק') מה הבעיה בשאילתה הבאה? כתבו תיקון קצר.

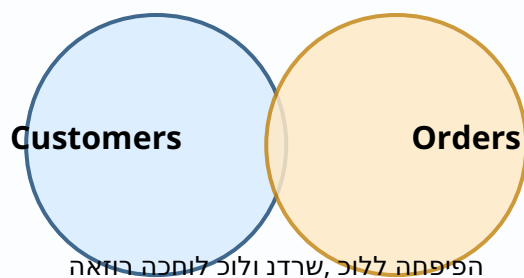
```
SELECT customer_id, COUNT(*)
FROM Orders;
```

חלק ג' - שאלות קוד קצרות - המשך

5. (5 נק') כתבו שאילתה שמחזירה לכל לקוח את מספר ההזמנות שלו. יש להחזיר גם לקוחות שאין להם הזמנות.

חלק ד' - שאלות ויזואליות ואיורים (15 נקודות)

1. (5 נק') התבוננו באיור הבא. איזה JOIN מתאים אם רוצים לקבל את כל הלקוחות, גם כאלה שלא ביצעו הזמנה?



1. INNER JOIN

2. LEFT JOIN מהטבלה Customers

3. RIGHT JOIN מהטבלה Products
4. CROSS JOIN

2. (5 נק') בתרשים הבא מוצגת טבלה לא מנורמלת. הסבירו בקצרה מה הבעיה ומה הייתם מפרידים לטבלאות.

product3	product2	product1	customer_phone	customer_name	order_id
Screen	Mouse	Keyboard	050-1111111	דנה כהן	501
		USB Cable	050-1111111	דנה כהן	502

3. (5 נק') לפי תרשים הביצוע, איפה כדאי לבדוק אפשרות לאינדקס?



חלק ה' - שאלות פתוחות (15 נקודות)

1. (5 נק') הסבירו במילים שלכם מהי נורמליזציה ולמה היא חשובה בתכנון מסד נתונים. ציינו יתרון אחד וחסרון אפשרי אחד.

2. (5 נק') נתונה השאילתה הבאה:

```
SELECT *
FROM Orders
WHERE YEAR(order_date) = 2025;
```

הסבירו מדוע היא עשויה להיות פחות יעילה, וכתבו גרסה טובה יותר מבחינת שימוש אפשרי באינדקס על order_date.

Products
3. (5 נק') כתבו שאילתה שמחזירה את 3 הקטגוריות עם סכום המכירות הגבוה ביותר. יש להשתמש בטבלאות OrderItems-1.

סוף המבחן - בהצלחה!

נספח תשובות מקוצר

מיועד לבדיקה עצמית לאחר פתרון המבחן.

חלק א' - אמריקאיות

1-ב, 2-ג, 3-א, 4-ב, 5-ב, 6-א, 7-ב, 8-א, 9-ב, 10-ג.

חלק ב' - נכון / לא נכון

1 נכון, 2 לא נכון, 3 נכון, 4 נכון, 5 לא נכון, 6 לא נכון, 7 נכון, 8 נכון.

חלק ג' - פתרונות אפשריים

```
-- 1
SELECT *
FROM Customers
WHERE city = 'Haifa'
ORDER BY join_date DESC;

-- 2
SELECT o.order_id, o.order_date, c.full_name
FROM Orders o
JOIN Customers c ON o.customer_id = c.customer_id;

-- 3
category = 'Books' AND price > 50

-- 4
-- GROUP BY עבור customer_id
SELECT customer_id, COUNT(*) AS orders_count
FROM Orders
GROUP BY customer_id;

-- 5
SELECT c.customer_id, c.full_name, COUNT(o.order_id) AS orders_count
FROM Customers c
LEFT JOIN Orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.full_name;
```

חלק ד' - תשובות מקוצרות

1. ב - LEFT JOIN מהטבלה Customers.

2. הבעיה: עמודות חוזרות למוצרים, כפילות פרטי לקוח, וקושי בהוספת מספר מוצרים משתנה. הפרדה אפשרית: Customers, Orders, Products, OrderItems.

3. כדאי לבדוק אינדקס על Orders.customer_id, כי זה השדה שעליו מתבצע הסינון.

חלק ה' - דגשים לתשובות

1. נורמליזציה מארגנת נתונים לטבלאות לפי ישויות וקשרים, מפחיתה כפילויות ומקטינה בעיות עדכון. חסרון אפשרי: יותר JOIN-ים ולעיתים שאילות מורכבות יותר.

2. הפעלת YEAR על עמודת התאריך עלולה למנוע שימוש יעיל באינדקס. עדיף:

```
SELECT order_id, customer_id, order_date, status
FROM Orders
WHERE order_date >= '2025-01-01'
      AND order_date < '2026-01-01';
```

3. פתרון אפשרי:

```
SELECT p.category, SUM(oi.quantity * oi.unit_price) AS total_sales
FROM OrderItems oi
JOIN Products p ON oi.product_id = p.product_id
GROUP BY p.category
ORDER BY total_sales DESC
LIMIT 3;
```